



Assessment 3 : Individual Programming Assignment

Lecturer: [Suirui ZHU](#)

Type: Individual

Unit Code: MIT102

Unit Name: MIT102 Operating Systems and Computer Architecture

Student Name: Anish Gurung

Student ID: aitc20251080

Declaration: I have read Academic Misconduct Policy and understand the consequences of committing academic misconduct as outlined in the policy. I declare this assessment is written in my own words.

Signature/s _



1. Introduction

This document provides complete technical documentation for the thread-safe hash table implemented for MIT102 Assessment 3. The hash table stores integers and is designed to be safely accessed by multiple concurrent threads without data corruption, race conditions, or deadlocks.

The implementation is written in Python 3 and uses the built-in threading module. No external libraries are required.

The project consists of three files:

- `hash_table.py` : the core hash table engine (no system calls)
- `tests.py` : automated test suite with 7 test groups and 18 assertions
- `cli.py` : interactive command-line interface for manual testing

2. System Architecture

2.1 Overview

The hash table maps integer keys to buckets using the formula:

$$\text{bucketNum} = x \% \text{numOfBuckets}$$

Each bucket maintains a singly linked list of integer nodes. Collisions are resolved by chaining : all values that hash to the same bucket are stored in that bucket's linked list.

2.2 Fine-Grained Locking

Thread safety is achieved by assigning one `threading.Lock` to each bucket. Threads operating on different buckets acquire different locks and therefore never block each other. Only threads accessing the same bucket will contend.

This is in contrast to a single global lock (coarse-grained locking), which forces all threads to queue regardless of which bucket they are accessing.



Locking Strategy	Concurrency	Contention
Single global lock	Low : all threads serialise	High : every operation waits
One lock per bucket (<i>this implementation</i>)	High : threads on different buckets run in parallel	Low : only same-bucket threads contend

2.3 Data Structures

Node (linked list node)

```
class Node:
    value: int # the stored integer
    next: Node # pointer to next node in bucket
```

HashTable

```
class HashTable:
    _buckets: list[Node] # head node per bucket
    _locks: list[threading.Lock] # one lock per bucket
    _num_buckets: int # total bucket count
    _initialized: bool # guards against uninitialised use
```

3. API Reference

All six public methods are thread-safe. Every method that accesses a bucket acquires that bucket's lock using a with statement, which guarantees the lock is released even if an exception occurs.

3.1 Hash_Init(numOfBuckets)

Initialises the hash table with the specified number of buckets. Must be called before any other method



Parameter	Type	Description
numOfBuckets	int	Number of buckets to create (must be > 0)

Behaviour:

- Allocates a list of None values (one head pointer per bucket)
- Creates one threading.Lock per bucket
- Prints a confirmation message
- Exits with error if numOfBuckets ≤ 0 or allocation fails

```
ht = HashTable()
```

```
ht.Hash_Init(16)
```

```
# Output: [Hash_Init] Initialized with 16 buckets (16 individual locks).
```

3.2 Hash_Insert(x)

Inserts integer x into the hash table. Prevents duplicate entries.

Return Value	Meaning
0	x was successfully inserted
-1	x already exists in the table : not inserted again

Behaviour:

- Calculates bucket: $x \% \text{numOfBuckets}$
- Acquires the bucket lock
- Walks the linked list to check for duplicates
- If duplicate found: releases lock, returns -1
- If no duplicate: prepends a new Node at the head of the list ($O(1)$), releases lock, returns 0

```
ht.Hash_Insert(42) # returns 0 (inserted)
```

```
ht.Hash_Insert(42) # returns -1 (duplicate)
```

```
ht.Hash_Insert(-7) # returns 0 (negative numbers supported)
```

```
ht.Hash_Insert(0) # returns 0 (zero supported)
```



3.3 Hash_Remove(x)

Removes integer x from the hash table.

Return Value	Meaning
0	x was found and removed
-1	x does not exist in the table

Behaviour:

- Calculates bucket: $x \% \text{numOfBuckets}$
- Acquires the bucket lock
- Walks the linked list tracking prev and current pointers
- If found: unlinks the node ($\text{prev.next} = \text{current.next}$), returns 0
- If not found: returns -1

```
ht.Hash_Remove(42) # returns 0 (removed)
```

```
ht.Hash_Remove(42) # returns -1 (not found)
```

3.4 Hash_CountElements()

Returns the total number of elements stored across all buckets.

Behaviour:

- Iterates over all buckets in order
- Acquires each bucket lock individually (not all at once)
- Counts nodes in the linked list while holding the lock
- Releases each lock before moving to the next bucket
- Returns the accumulated total

```
count = ht.Hash_CountElements() # e.g. returns 3
```

3.5 Hash_CountBucketElements(bucketNumber)

Returns the number of elements in the specified bucket. Exits with an error if bucketNumber is out of



range.

Parameter	Type	Description
bucketNumber	int	Bucket index (0 to numOfBuckets - 1)

```
count=ht.Hash_CountBucketElements(3) # count nodes in bucket 3
```

3.6 Hash_Dump()

Prints the full contents of the hash table to stdout. Acquires each bucket lock while reading, then releases it before printing the next.

```
=====
HASH TABLE DUMP (8 buckets)
=====
```

```
Bucket[ 0]: (empty)
```

```
Bucket[ 2]: 18 -> 10 -> NULL
```

```
Bucket[ 3]: 99 -> NULL
=====
```

```
Total elements: 3
=====
```

4. Thread Safety Analysis

4.1 Lock Acquisition

Every method wraps all linked list operations inside a with self._locks[bucket] block:

```
with self._locks[bucket]:
```

```
    # All reads and writes to this bucket's linked list
```

```
    # happen here, atomically from other threads' perspective
```

```
    current = self._buckets[bucket]
```

```
    while current is not None:
```

```
        if current.value == x:
```

```
            return -1
```

```
        current = current.next
```

```
    new_node = Node(x)
```

```
    new_node.next = self._buckets[bucket]
```



```
self._buckets[bucket] = new_node
```

4.2 Race Conditions Prevented

Scenario	Without Locking	With Fine-Grained Locking
Two threads insert same value simultaneously	Both pass duplicate check : two copies inserted	Second thread waits; sees duplicate, returns -1
Thread reads while another writes same bucket	May read partially updated list (torn read)	Reader waits for writer to release lock first
Two threads remove same node	Double-free; second remove corrupts the list	Second thread finds node already gone, returns -1
Count while inserts/removes happening	May count nodes mid-modification	Each bucket locked during count; consistent snapshot

4.3 Deadlock Avoidance

Deadlock requires a circular wait. This implementation avoids it because:

- Each method acquires at most one bucket lock at a time
- Hash_CountElements() acquires and releases each lock sequentially : it never holds two locks simultaneously
- There is no circular dependency possible with a single-lock-at-a-time strategy



5. Error Handling

All errors are written to stderr and the program exits via `sys.exit(1)`.

Error Condition	Method	Behaviour
Hash table not initialised	All methods	Prints error, calls <code>sys.exit(1)</code>
<code>numOfBuckets <= 0</code>	<code>Hash_Init</code>	Prints error, calls <code>sys.exit(1)</code>
Memory allocation failure	<code>Hash_Init</code> , <code>Hash_Insert</code>	Prints error, calls <code>sys.exit(1)</code>
<code>bucketNumber</code> out of range	<code>Hash_CountBucketElements</code>	Prints error, calls <code>sys.exit(1)</code>
<code>x</code> not found (expected)	<code>Hash_Remove</code>	Returns -1 (not an error)
<code>x</code> already exists (expected)	<code>Hash_Insert</code>	Returns -1 (not an error)

6. Command-Line Interface

The interactive CLI (`cli.py`) allows manual testing of all hash table operations. System calls (`input`, `print`) are permitted in this file.

6.1 How to Run

python [cli.py](#)



6.2 Commands

Command	Description	Example
init <n>	Initialise with n buckets	init 16
insert <x>	Insert integer x	insert 42
remove <x>	Remove integer x	remove 42
count	Total element count	count
bucket <n>	Elements in bucket n	bucket 3
dump	Print full hash table	dump
help	Show all commands	help
quit	Exit the CLI	quit

6.3 Example Session

ht> init 8

[Hash_Init] Initialized with 8 buckets (8 individual locks).

ht> insert 10

Inserted.

ht> insert 10

Already exists (-1).

ht> insert 18

Inserted.

ht> count

Total elements: 2

ht> bucket 2



Bucket 2 elements: 2

ht> dump

Bucket[2]: 18 -> 10 -> NULL

ht> remove 10

Removed.

ht> remove 10

Not found (-1).

ht> quit

7. Test Suite

7.1 How to Run

python [tests.py](#)

7.2 Test Groups

Group	Name	Assertions
1	Basic Functionality (single-threaded)	9
2	Concurrent Inserts	1
3	Concurrent Removes	1
4	Mixed Concurrent Operations	2
5	Duplicate Detection Under Concurrency	3
6	Count Accuracy Under Concurrent Modifications	1
7	Stress Test	1



7.3 All Test Cases

#	Test	What It Verifies
1	Insert returns 0	Successful insert return value is 0
2	Duplicate returns -1	Re-inserting the same value returns -1 without modifying the table
3	CountElements after 4 inserts == 4	Total count is accurate after multiple inserts
4	Remove returns 0	Successful removal return value is 0
5	Remove non-existent returns -1	Removing a missing value returns -1
6	CountElements after remove == 3	Count decrements correctly after removal
7	CountBucketElements accuracy	Per-bucket count is correct (10 and 18 both hash to bucket 2)
8	Negative numbers	Negative integers are inserted and removed correctly
9	Zero	Zero is inserted and removed correctly
10	Concurrent inserts — no data loss	20 threads × 50 unique inserts = 1,000 elements with no losses
11	Concurrent removes — all succeed	20 threads × 10 removes leave the table empty
12	Mixed ops — final count correct	Inserting 100–199 while removing 0–99 leaves exactly 100 elements
13	No negative counts during reads	Hash_CountElements never returns < 0 under concurrent modification



#	Test	What It Verifies
14	Only one insert wins race	50 threads inserting the same value : exactly 1 succeeds
15	Correct duplicate count	The other 49 threads all correctly receive -1
16	Table holds exactly 1 copy	Hash_CountElements returns 1 after the 50-thread race
17	Count matches tracked inserts	Hash_CountElements equals the sum of all return-0 insert calls
18	Stress test correct final count	50 threads $\times$ 200 random ops produce the correct final element count

7.4 Stress Test Results

Threads : 50

Ops per thread : 200

Total ops : 10,000

Elapsed time : ~0.022s

Throughput : ~462,000 ops/sec

Result : PASS (expected == actual)



8. File Reference

File	System Calls	Purpose
hash_table.py	None	HashTable class with all six required methods
tests.py	print()	Automated test suite — 7 groups, 18 assertions
cli.py	input(), print()	Interactive REPL for manual testing
README.md	N/A	Quick-start guide and API summary